



KAPITEL

11

Mehr über Vererbung

Lernziele

Zentrale Konzepte in diesem Kapitel: Methoden-Polymorphie, statischer und dynamischer Typ, Überschreiben von Methoden, dynamische Methodensuche

Java-Konstrukte in diesem Kapitel: `super` (in Methoden), `toString`, `protected`, `instanceof`

Im letzten Kapitel wurden die Kernkonzepte der Vererbung anhand des *Netzwerk*-Beispiels eingeführt. Daneben gibt es aber noch etliche wichtige Details zu untersuchen. Vererbung ist von grundlegender Bedeutung für das Verständnis und die Programmierung mit objektorientierten Sprachen; deshalb ist es für die weiteren Abschnitte sehr wichtig, Vererbung bis in die Details hinein verstanden zu haben.

In diesem Kapitel werden wir weiter mit *Netzwerk* arbeiten, um uns die wichtigsten der übrigen Aspekte rund um Vererbung und Polymorphie anzueignen.

11.1 Das Problem: die Methode zum Anzeigen

Als Sie mit den *Netzwerk*-Beispielen in Kapitel 10 experimentiert haben, haben Sie möglicherweise festgestellt, dass die zweite Version – die mit Vererbung – ein Problem hat: Die Methode **anzeigen** zeigt nicht alle Details einer Einsendung an.

Betrachten wir ein Beispiel. Nehmen wir an, wir erzeugen ein **NachrichtenEinsendung**- und ein **FotoEinsendung**-Objekt mit den folgenden Daten:

Die Nachrichteneinsendung:

Leonardo da Vinci

Hatte heute Morgen eine grossartige Idee. Doch jetzt habe ich vergessen, was es war.

Es hatte irgendwas mit Fliegen zu tun ...

vor 40 Sekunden. 2 Person(en) gefällt dies.

Keine Kommentare.

Die Fotoeinsendung:

Alexander Graham Bell

[experiment.jpg]

Ich denke, ich werde dies Ding „Telefon“ nennen.

Vor 12 Minuten. 4 Person(en) gefällt dies.

Keine Kommentare.

Wenn wir diese in den Newsfeed¹ eintragen und dann die erste Version der Methode **zeigen** aufrufen (die ohne Vererbung), wird Folgendes ausgegeben:

Leonardo da Vinci

Hatte heute Morgen eine grossartige Idee.

Doch jetzt habe ich vergessen, was es war. Es hatte irgendwas mit Fliegen zu tun ...

vor 40 Sekunden. 2 Person(en) gefällt dies.

Keine Kommentare.

Alexander Graham Bell

[experiment.jpg]

Ich denke, ich werde dies Ding 'Telefon' nennen.

vor 12 Minuten. 4 Person(en) gefällt dies.

Keine Kommentare.

Die Formatierung ist zwar nicht besonders ansprechend (da uns in der Textkonsole keine Formatierungsoptionen zur Verfügung stehen), aber alle Informationen sind vorhanden und wir können uns vorstellen, wie die Methode **zeigen** später angepasst werden kann, um die Daten in einer anderen Benutzerschnittstelle netter zu formatieren.

Vergleichen Sie dies mit der zweiten Version von *Netzwerk* (die mit Vererbung), die lediglich ausgibt:

Leonardo da Vinci

Vor 40 Sekunden - 2 Person(en) gefällt dies

Keine Kommentare

Alexander Graham Bell

Vor 12 Sekunden - 4 Person(en) gefällt dies

Keine Kommentare

Wir sehen, dass der Text der Nachrichteneinsendung sowie der Bilddateiname und die Überschrift der Fotoeinsendung fehlen. Der Grund ist einfach. Die Methode **anzeigen** ist in dieser Version in der Klasse **Einsendung** implementiert, nicht in **NachrichtenEinsendung** und **FotoEinsendung** (Abbildung 11.1). In der Methode in **Einsendung** sind jedoch nur die Datenfelder verfügbar, die auch in **Einsendung** definiert sind. Wenn wir in der Methode **anzeigen** in **Einsendung** versuchen würden, auf das Datenfeld **nachricht** von **NachrichtenEinsendung** zuzugreifen, würde es eine Fehlermeldung geben. Dies illustriert das wichtige Prinzip, dass Vererbung als eine Einbahnstraße betrachtet werden sollte: **NachrichtenEinsendung** erbt die Datenfelder von **Einsendung**, aber **Einsendung** weiß nichts über die Datenfelder ihrer Subklassen.

¹ Der Text für die Nachrichteneinsendung ist eine zweizeilige Zeichenkette. Sie können einen mehrzeiligen Text in eine Zeichenkette eingeben, indem Sie das Zeichen `\n` als Zeilenumbruch einfügen.

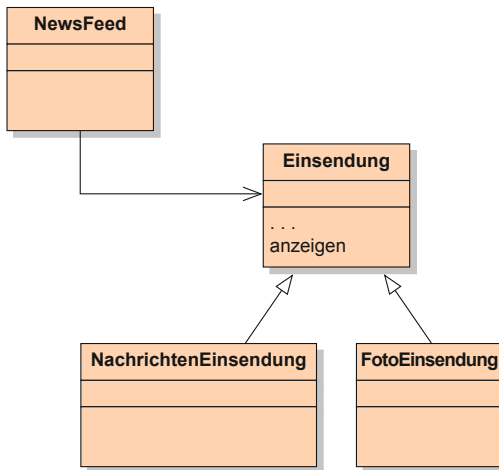


Abbildung 11.1
Ausgabe, Version 1:
die Methode **anzeigen**
in der Superklasse.

11.2 Statischer und dynamischer Typ

Der Versuch, eine vollständig polymorphe Methode **anzeigen** zu entwickeln, führt uns in eine Diskussion über *statische* und *dynamische Typen* und *Methodensuche*. Aber fangen wir langsam an.

Ein erster Versuch zur Lösung des Problems könnte sein, die Methode **anzeigen** in die Subklassen zu verschieben (Abbildung 11.2). Auf diese Weise könnten die beiden Methoden **anzeigen**, die dann zu den Klassen **NachrichtenEinsendung** und **FotoEinsendung** gehören, jeweils auf die Datenfelder von **NachrichtenEinsendung** und **FotoEinsendung** zugreifen. Sie könnten auch auf die geerbten Datenfelder zugreifen, indem sie sondierende Methoden aus der Klasse **Einsendung** aufrufen. Das sollte diesen Methoden ermöglichen, alle notwendigen Details auszugeben. Setzen Sie diesen Ansatz um, indem Sie die folgende Übung durchführen.

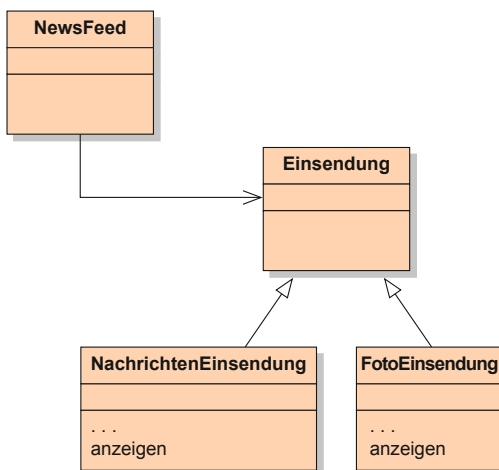


Abbildung 11.2
Ausgabe, Version 2:
die Methode **anzeigen**
in den Subklassen.

Übung 11.1 Öffnen Sie Ihre letzte Version des *Netzwerk*-Projekts. (Sie können *Netzwerk-V2* benutzen, wenn Sie noch kein eigenes Projekt haben.) Entfernen Sie die Methode **anzeigen** aus der Klasse **Einsendung** und verschieben Sie sie in die Klassen **FotoEinsendung** und **NachrichtenEinsendung**. Übersetzen Sie. Was beobachten Sie?

Wenn wir die Methode **anzeigen** einfach in die Unterklassen verschieben, stellen wir fest, dass sich das Projekt nicht mehr übersetzen lässt. Es gibt zwei fundamentale Probleme:

- Wir bekommen Fehlermeldungen in den Klassen **NachrichtenEinsendung** und **FotoEinsendung**, weil wir dort nicht auf die Datenfelder der Superklasse zugreifen können.
- Wir bekommen eine Fehlermeldung in der Klasse **NewsFeed**, weil dort die Methode **anzeigen** nicht gefunden werden kann.

Der Grund für die erste Fehlerart ist, dass die Datenfelder in **Einsendung** als privat deklariert und deshalb für alle anderen Klassen nicht zugreifbar sind – inklusive aller Subklassen. Da wir die Kapselung nicht durchbrechen wollen, indem wir diese Datenfelder öffentlich deklarieren, ist die nächstliegende Lösung, die Datenfelder über öffentliche sondierende Methoden zugreifbar zu machen. (Hinweis: In Abschnitt 11.9 werden wir noch eine weitere Zugriffsart vorstellen, die speziell für die Beziehung zwischen einer Super- und ihren Subklassen definiert wurde.)

Der zweite Fehler erfordert eine etwas ausführlichere Erklärung, die wir im folgenden Abschnitt geben.

11.2.1 Aufrufe von **anzeigen** in **NewsFeed**

Wir untersuchen zuerst das Problem, dass **anzeigen** in der Klasse **NewsFeed** nicht aufgerufen werden kann. Die betroffenen Zeilen im Quelltext der Klasse **NewsFeed** sind:

```
for(Einsendung einsendung : einsendungen) {
    einsendung.anzeigen();
    System.out.println();
}
```

Die **for-each**-Anweisung fragt jede **Einsendung** aus der Sammlung ab; die erste Anweisung im Schleifenrumpf versucht, die Methode **anzeigen** an der **Einsendung** aufzurufen. Der Compiler meldet, dass er die Methode **anzeigen** nicht finden kann.

Einerseits erscheint dies logisch: **Einsendung** hat tatsächlich keine Methoden **anzeigen** mehr (Abbildung 11.2).

Andererseits erscheint es unlogisch und ist ärgerlich. Wir wissen, dass jedes **Einsendung**-Objekt, das in der Datenbank eingetragen ist, entweder ein **NachrichtenEinsendung**- oder ein **FotoEinsendung**-Objekt ist, und beide haben eine Methode **anzeigen**. Also könnte ein Aufruf **Einsendung.anzeigen()** eigentlich funktionieren. Egal, ob es ein **NachrichtenEinsendung**- oder ein **FotoEinsendung**-Objekt ist, wir wissen, dass eine Methode **anzeigen** vorhanden ist.

Um im Detail zu verstehen, warum dies nicht funktioniert, sollten wir uns die Typen näher ansehen. Betrachten wir folgende Anweisung:

```
Auto a1 = new Auto();
```

Wir sagen: Der Typ von **a1** ist **Auto**. Bevor wir Vererbung kennengelernt haben, hat es keine Rolle gespielt, ob wir mit „Typ von **a1**“ gemeint haben „der Typ der Variablen **a1**“ oder „der Typ des Objekts, das in **a1** gespeichert ist“. Das war nicht wichtig, weil der Typ der Variablen und der Typ des Objekts immer gleich waren.

Da wir jetzt das Konzept von Subtypen kennen, müssen wir etwas präziser werden. Betrachten Sie folgende Anweisung:

```
Fahrzeug f1 = new Auto();
```

Was ist der Typ von **f1**? Das hängt davon ab, was wir mit „Typ von **f1**“ meinen. Der Typ der Variablen **f1** ist **Fahrzeug**; der Typ des Objekts, auf das **f1** verweist, ist jedoch **Auto**. Durch die Regeln von Subtypen und Ersetzbarkeit haben wir hier eine Situation, in der der Typ der Variablen und der Typ des dahinterliegenden Objekts nicht mehr gleich sind.

Um besser über diesen Tatbestand reden zu können, führen wir einige Begriffe ein:

- Wir nennen den deklarierten Typ einer Variablen ihren *statischen Typ*, denn er wird im Quelltext deklariert – der statischen Repräsentation eines Programms.
- Wir nennen den Typ des Objekts, auf das eine Variable verweist, ihren *dynamischen Typ*, denn dieser hängt von Zuweisungen zur Laufzeit ab – vom dynamischen Verhalten des Programms.

Somit können wir die obige Anweisung jetzt präziser beschreiben: Der statische Typ von **f1** ist **Fahrzeug**, der dynamische Typ von **f1** ist **Auto**. Wir können nun auch unser Problem mit dem Aufruf von **anzeigen** in der Klasse **NewsFeed** umformulieren. Beim Aufruf

```
einsendung.anzeigen();
```

ist der statische Typ von **einsendung** hier **Einsendung**, während der dynamische Typ entweder **NachrichtenEinsendung** oder **FotoEinsendung** ist (Abbildung 11.3). Wir wissen nicht, welcher von beiden es ist, wenn wir davon ausgehen, dass sowohl **NachrichtenEinsendung**- als auch **FotoEinsendung**-Objekte in den Feed eingetragen wurden.



Der Compiler meldet einen Fehler, weil bei der Typüberprüfung ausschließlich der statische Typ berücksichtigt wird. Da der dynamische Typ häufig erst zur Laufzeit bekannt ist, hat der Compiler gar keine andere Wahl: Zur Übersetzungszeit kann er nur den statischen Typ berücksichtigen. Der statische Typ von **einsendung** ist **Einsendung**, und **Einsendung** verfügt nicht über eine Methode **anzeigen**. Das Verhalten des Compilers ist vernünftig, denn er hat keine Garantie, dass *alle* Subklassen von **Einsendung** tatsächlich eine Methode **anzeigen** definieren; in der Praxis ist das schlicht nicht prüfbar.

Konzept

Der **statische Typ** einer Variablen **v** ist der Typ, mit dem die Variable im Quelltext der Klasse deklariert wurde.

Konzept

Der **dynamische Typ** einer Variablen **v** ist der Typ des Objekts, das zurzeit in der Variablen **v** gehalten wird.

Abbildung 11.3

Eine Variable vom Typ **Einsendung** enthält ein Objekt vom Typ **FotoEinsendung**.

Mit anderen Worten: Damit dieser Aufruf funktioniert, benötigt **Einsendung** eine Methode **anzeigen**; damit sind wir wieder bei unserem ursprünglichen Problem, scheinbar, ohne einen Schritt weitergekommen zu sein.

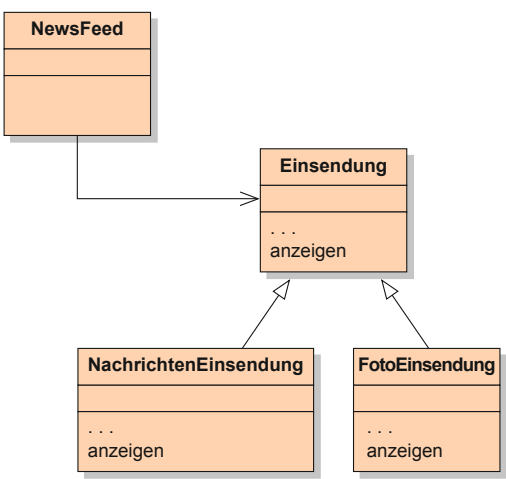
Übung 11.2 Fügen Sie in Ihrem *Netzwerk*-Projekt wieder eine Methode **anzeigen** in die Klasse **Einsendung** ein. Lassen Sie im Rumpf der Methode vorläufig nur den Benutzernamen ausgeben. Modifizieren Sie dann die **anzeigen**-Methoden in den Klassen **NachrichtenEinsendung** und **FotoEinsendung** so, dass die **NachrichtenEinsendung**-Version nur die Nachricht ausgibt und die **FotoEinsendung**-Version nur den Überschrift. Dies beseitigt die oben aufgetretenen Fehler (auf die wir noch zurückkommen werden).

Sie sollten nun eine Situation wie in Abbildung 11.4 haben: eine Methode **anzeigen** in drei Klassen. Übersetzen Sie Ihr Projekt. (Wenn es Fehler gibt, sollten Sie diese beseitigen; der Entwurf sollte funktionieren.)

Bevor Sie den neuen Entwurf ausführen, sollten Sie vorhersagen, welche der **anzeigen**-Methoden aufgerufen wird, wenn Sie die Methode **zeigen** des News-feed aufrufen.

Versuchen Sie es: Tragen Sie ein **NachrichtenEinsendung** und ein **FotoEinsendung**-Objekt in die **Datenbank** ein und rufen Sie die Methode **auflisten** auf. Welche **anzeigen**-Methode wurde ausgeführt? War Ihre Vorhersage korrekt? Versuchen Sie Ihre Beobachtungen zu erklären.

Abbildung 11.4
Ausgabe, Version 3:
eine Methode
anzeigen in den
Subklassen und der
Superklasse.



11.3 Überschreiben von Methoden

Als Nächstes diskutieren wir den Entwurf, in dem sowohl die Super- als auch die beiden Subklassen eine Methode **anzeigen** haben (Abbildung 11.4). Der Kopf aller drei **anzeigen**-Methoden ist exakt gleich.

Listing 11.1 zeigt die relevanten Details der Quelltexte der drei Klassen. Die Klasse **Einsendung** hat eine Methode **anzeigen**, die alle Datenfelder der Klasse **Ein-**

sendung ausgibt (die sowohl für Nachrichteneinsendungen als auch für Fotoeinsendungen gelten), die Subklassen **NachrichtenEinsendung** und **FotoEinsendung** haben Methoden, die die Daten der jeweiligen Klasse ausgeben.

```
public class Einsendung
{
    ...

    public void anzeigen()
    {
        System.out.println(benutzername);
        System.out.print(zeitString(zeitstempel));

        if(gefielWieOft > 0) {
            System.out.println(" - " + gefielWieOft +
                " Person(en) gefaellt dies.");
        }
        else {
            System.out.println();
        }

        if(kommentare.isEmpty()) {
            System.out.println(" Keine Kommentare.");
        }
        else {
            System.out.println(" " + kommentare.size() +
                " Kommentar(e). Zum Einsehen hier klicken.");
        }
    }
}
```

```
public class NachrichtenEinsendung extends Einsendung
{
    ...

    public void anzeigen()
    {
        System.out.println(nachricht);
    }
}
```

```
public class FotoEinsendung extends Einsendung
{
    ...

    public void anzeigen()
    {
        System.out.println(" [" + dateiname + "]");
        System.out.println(" " + ueberschrift);
    }
}
```

Listing 11.1

Quelltext der **anzeigen**-Methoden in allen drei Klassen.

Konzept

Überschreiben einer Methode:

Eine Subklasse kann die Implementierung einer Methode überschreiben. Dazu deklariert die Subklasse eine Methode mit der gleichen Signatur wie in der Superklasse, implementiert diese jedoch mit einem anderen Rumpf. Die überschreibende Methode wird dann bei Aufrufen an Objekte der Unterklasse vorgezogen.

Dieser Entwurf funktioniert etwas besser. Er lässt sich übersetzen und kann ausgeführt werden, auch wenn er noch nicht perfekt ist. Eine Implementierung dieses Entwurfs liefert das Projekt *Netzwerk-V3*. (Wenn Sie die **Übung 11.2** durchgeführt haben, dann haben Sie bereits eine ähnliche Umsetzung dieses Entwurfs in Ihrem eigenen Projekt.)

Die Technik, die hier angewendet wird, nennt sich *Überschreiben* (und wird manchmal auch als *Redefinieren* bezeichnet). Überschreiben tritt auf, wenn eine Methode in einer Superklasse definiert ist (die Methode **anzeigen** in der Klasse **Einsendung** in diesem Beispiel) und eine Methode mit der exakt gleichen Signatur in einer Subklasse definiert wird. Man kann **@Override** vor die Version der Subklasse setzen, um deutlich zu machen, dass hier eine neue Version einer geerbten Methode definiert wird.

In einer solchen Situation haben die Objekte der Subklassen zwei Methoden mit gleichem Namen und gleicher Signatur: eine aus der Superklasse geerbte und eine aus der Subklasse. Welche wird ausgeführt, wenn wir die Methode aufrufen?

11.4 Dynamische Methodensuche

Wir erleben eine kleine Überraschung, wenn wir die Methode **zeigen** der Klasse **NewsFeed** aufrufen. Wenn wir wie in Abschnitt 11.1 einige Objekte erzeugen und eintragen, dann liefert die Methode **zeigen** in der neuen Version folgende Ausgabe:

```
Ich hatte eine großartige Idee heute Morgen.  
Doch jetzt habe ich vergessen, was es war. Es hatte irgendwas mit  
Fliegen zu tun ...
```

```
[experiment.jpg]  
Ich denke, ich werde dies Ding 'Telefon' nennen.
```

Wir können an dieser Ausgabe erkennen, dass die **anzeigen**-Methoden der Klassen **NachrichtenEinsendung** und **FotoEinsendung** aufgerufen werden, aber nicht die der Klasse **Einsendung**.

Das mag auf den ersten Blick seltsam erscheinen. Unsere Untersuchung in Abschnitt 11.2 hat ergeben, dass der Compiler auf eine Methode **anzeigen** in der Klasse **Einsendung** besteht – die Methoden in den Subklassen haben nicht ausgereicht. Der letzte Versuch hat nun gezeigt, dass diese Methode aus der Klasse **Einsendung** überhaupt nicht ausgeführt wird, stattdessen aber die der Subklassen. Zusammengefasst:

- Die Typüberprüfung berücksichtigt den statischen Typ, aber zur Laufzeit werden die Methoden des dynamischen Typs ausgeführt.

Das ist eine sehr wichtige Aussage. Um sie besser zu verstehen, sehen wir uns genauer an, wie Methoden aufgerufen werden. Dieser Mechanismus wird *Methodensuche* (*method lookup*), *Methodenbindung* (*method binding*) oder *Methodenauswahl* (*method dispatch*) genannt. Wir werden im Weiteren den Begriff „Methodensuche“ verwenden.

Wir beginnen mit einem einfachen Szenario für eine Methodensuche. Nehmen Sie an, wir haben ein Objekt der Klasse **FotoEinsendung**, das in einer Variablen **v1** vom statischen Typ **FotoEinsendung** gespeichert ist (Abbildung 11.5). Die Klasse **FotoEinsendung** hat eine Methode **anzeigen** und keine explizit deklarierte Superklasse. Das ist eine sehr einfache Situation – weder Vererbung noch Polymorphie sind beteiligt. Wir führen nun folgende Anweisung aus:

```
v1.anzeigen();
```

Wenn diese Anweisung ausgeführt wird, erfolgt der Aufruf der Methode **anzeigen** in folgenden Schritten:

- 1** Auf die Variable **v1** wird zugegriffen.
- 2** Das in der Variablen gespeicherte Objekt wird gefunden (indem der Referenz gefolgt wird).
- 3** Die Klasse dieses Objekts wird gefunden (indem der Referenz „ist Instanz von“ gefolgt wird).

- 4** Die Implementierung der Methode **anzeigen** wird in der Klasse gefunden und ausgeführt.

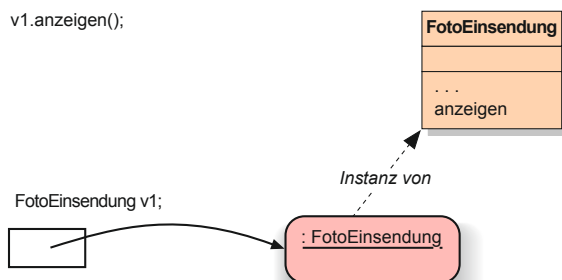


Abbildung 11.5
Methodensuche mit
einem einfachen
Objekt.

Bisher ist alles sehr gradlinig und nicht überraschend.

Als Nächstes betrachten wir eine Methodensuche mit Vererbung. Das Szenario ist ähnlich, aber diesmal hat die Klasse **FotoEinsendung** eine Superklasse **Einsendung** und die Methode **anzeigen** ist ausschließlich in der Superklasse definiert (Abbildung 11.6).

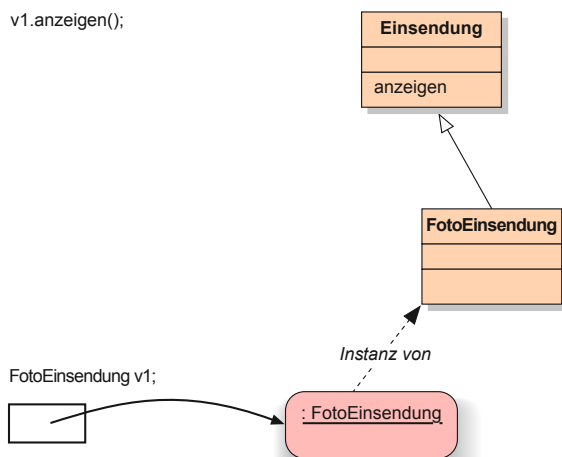


Abbildung 11.6
Methodensuche mit
Vererbung.

Wir führen dieselbe Anweisung aus. Der Aufruf der Methode beginnt ähnlich wie oben, die ersten drei Schritte sind unverändert. Aber dann geht es anders weiter:

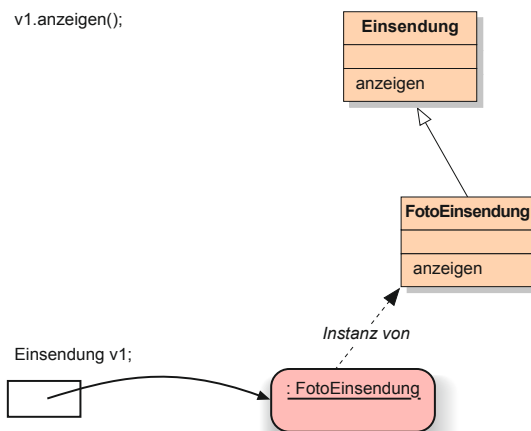
- 4** In der Klasse **FotoEinsendung** wird keine Methode **anzeigen** gefunden.
- 5** Weil keine passende Methode gefunden wird, wird die Superklasse nach einer passenden Methode durchsucht. Wenn auch dort die Methode nicht gefunden wird, wird die nächste Superklasse (wenn es noch eine gibt) durchsucht. Dies setzt sich durch die Vererbungshierarchie nach oben zur Klasse **Object** hin fort, bis eine Methode gefunden wird. Beachten Sie, dass zur Laufzeit garantiert eine passende Methode gefunden wird, sonst hätte die Klasse nicht übersetzt werden können.
- 6** In unserem Beispiel wird die Methode **anzeigen** in der Klasse **Einsendung** gefunden und ausgeführt.

Dieses Szenario beschreibt, wie Objekte Methoden erben. Jede Methode, die in einer Superklasse gefunden werden kann, kann auf einem Objekt einer Subklasse aufgerufen werden und wird korrekt gefunden und ausgeführt.

Als Nächstes kommen wir zum spannendsten Szenario: der Methodensuche mit einer polymorphen Variablen und einer überschriebenen Methode (Abbildung 11.7). Die Situation ist wieder der vorherigen recht ähnlich, aber es gibt zwei Änderungen:

- Der deklarierte Typ der Variablen **v1** ist nun **Einsendung**, nicht **FotoEinsendung**.
- Die Methode **anzeigen** ist definiert in der Klasse **Einsendung** und redefiniert (überschrieben) in der Klasse **FotoEinsendung**.

Abbildung 11.7
Methodensuche mit
Polymorphie und
Überschreiben.



Dieses Szenario ist das wichtigste, um das Verhalten in unserer kleinen *Netzwerk*-Anwendung zu verstehen und eine Lösung für unser Problem der **anzeigen**-Methoden zu finden.

Die Schritte, in denen die Methode ausgeführt wird, sind exakt die gleichen Schritte 1 bis 4 wie im ersten Szenario. Lesen Sie diese noch einmal.

Einige Beobachtungen sind hervorzuheben:

- Es gibt keine besonderen Suchregeln für die Methodensuche, wenn der dynamische Typ nicht gleich dem statischen Typ ist. Das beobachtete Verhalten entspricht genau den allgemeinen Regeln.
- Welche Methode zuerst gefunden und ausgeführt wird, hängt vom dynamischen Typ ab, nicht vom statischen Typ. Anders gesagt: Die Tatsache, dass der deklarierte Typ der Variablen **v1** hier **Einsendung** ist, spielt keine Rolle. Die Instanz, mit der wir hier umgehen, ist von der Klasse **FotoEinsendung** – das ist das Einzige, worauf es ankommt.
- Überschreibende Methoden in Subklassen haben Vorrang vor Methoden aus Superklassen. Da die Methodensuche in der dynamischen Klasse der Instanz beginnt (am unteren Ende der Vererbungshierarchie), wird die letzte Redefinition einer Methode zuerst gefunden und auch entsprechend ausgeführt.
- Wenn eine Methode überschrieben ist, dann wird nur die letzte Version (die am weitesten unten stehende in der Vererbungshierarchie) ausgeführt. Versionen der gleichen Methode in den Superklassen werden nicht automatisch ebenfalls ausgeführt.

Dies erklärt das Verhalten, das wir in unserem *Netzwerk*-Projekt beobachten. Bei der Ausgabe der Einsendungen werden nur die **anzeigen**-Methoden aus den Subklassen (**NachrichtenEinsendung** und **FotoEinsendung**) ausgeführt, und es kommt zu unvollständigen Ausgaben. Im nächsten Abschnitt besprechen wir, wie wir dies beheben können.

11.5 super-Aufrufe in Methoden

Nachdem wir nun detailliert verstehen, wie überschriebene Methoden ausgeführt werden, können wir die Lösung des Problems nachvollziehen. Man erkennt leicht, dass es für die gegebene Situation das Eleganteste wäre, wenn für jeden Aufruf der Methode **anzeigen** für beispielsweise ein **FotoEinsendung**-Objekt sowohl die **anzeigen**-Methode aus **Einsendung** als auch die **anzeigen**-Methode aus **FotoEinsendung** ausgeführt würde. Dann würden alle Details ausgegeben werden. (Eine alternative Lösung werden wir später noch besprechen.)

Das ist tatsächlich recht einfach zu erreichen. Wir können das **super**-Konstrukt benutzen, dem wir bereits im Zusammenhang mit Konstruktoren in Kapitel 10 begegnet sind. Listing 11.2 zeigt die Idee anhand der **anzeigen**-Methode in der Klasse **NachrichtenEinsendung**.

```
public void anzeigen()
{
    super.anzeigen();
    System.out.println(" [" + dateiname + "]");
    System.out.println(" " + ueberschrift);
}
```

Listing 11.2

Redefinierende Methode mit **super**-Aufruf.

Wenn nun **anzeigen** an einem **FotoEinsendung**-Objekt aufgerufen wird, wird zuerst die Methode **anzeigen** aus der Klasse **FotoEinsendung** ausgeführt. Als erste Anweisung ruft diese Methode die Methode **anzeigen** der Superklasse auf, die wiederum die allgemeinen Informationen über eine Sendung ausgibt. Wenn der Kontrollfluss aus der Superklasse zurückkehrt, sorgen die restlichen Anweisungen in der Methode der Subklasse für die Ausgabe der spezifischen Daten der **FotoEinsendung**-Klasse.

Drei Details sind hier zu beachten:

- Im Gegensatz zu **super**-Aufrufen in Konstruktoren wird der Name der Methode der Superklasse explizit genannt. Ein **super**-Aufruf in einer Methode hat immer folgende Form:

```
super.Methodenname ( Parameterliste )
```

Die Parameterliste kann dabei natürlich leer sein.

- Ebenfalls im Gegensatz zur Regel für **super**-Aufrufe in Konstruktoren kann der **super**-Aufruf in einer Methode an jeder beliebigen Stelle in der Methode erfolgen. Er muss nicht die erste Anweisung sein.
- Im Gegensatz zu **super**-Aufrufen in Konstruktoren muss kein **super**-Aufruf erfolgen und es wird auch nicht automatisch ein Aufruf generiert; er ist optional. Auf diese Weise ergibt sich das Standardverhalten, dass eine überschreibende Methode die entsprechende Version aus der Superklasse komplett verdeckt (überschreibt).

Übung 11.3 Fügen Sie in Ihrer neuesten Version des *Netzwerk*-Projekts den **super**-Aufruf in die **anzeigen**-Methoden ein. Testen Sie. Verhält sich die Anwendung wie erwartet? Sehen Sie Probleme mit dieser Lösung?

In diesem Zusammenhang sei noch einmal auf die Ausführungen von Abschnitt 10.6 verwiesen: Grundsätzlich kann eine Subklasse auf alle nicht privaten Elemente ihrer Superklasse ohne spezielle Syntax direkt zugreifen. Ein Zugriff über **super** ist nur dann erforderlich, wenn die Superklassen-Version einer *überschriebenen* Methode aufgerufen werden soll.

Wenn Sie **Übung 11.3** abgeschlossen haben, werden Sie feststellen, dass diese Lösung zwar funktioniert, aber noch nicht perfekt ist. Sie gibt alle Details aus, aber nicht in der gewünschten Reihenfolge. Dieses werden wir weiter hinten im Kapitel beheben.

11.6 Methoden-Polymorphie

Konzept

Methoden-Polymorphie:

Methodenaufrufe in Java sind polymorph. Derselbe Methodenaufruf kann zu unterschiedlichen Zeitpunkten verschiedene Methoden aufrufen, abhängig vom dynamischen Typ der Variablen, mit der der Aufruf durchgeführt wird.

In den vorigen Abschnitten haben wir eine weitere Form der Polymorphie kennengelernt. Diese neue Form ist unter dem Begriff *polymorphe Methodensuche* (oder kurz *Methoden-Polymorphie*) bekannt.

Erinnern Sie sich, die definierende Eigenschaft einer polymorphen Variablen besteht darin, dass sie auf Objekte unterschiedlicher Typen verweisen kann (jede Objektvariable ist in Java potenziell polymorph). Auf ähnliche Weise sind Methodenaufrufe in Java polymorph, denn ihr Aufruf kann zur Ausführung unterschiedlicher Methoden zu unterschiedlichen Zeitpunkten führen. Beispielsweise kann der Aufruf

```
einsendung.anzeigen();
```

zu einem Zeitpunkt die **anzeigen**-Methode einer **NachrichtenEinsendung** aufrufen und zu einem anderen die **anzeigen**-Methode einer **FotoEinsendung**, je nach referenziertem Objekt, also je nach dynamischem Typ der Variablen **sendung**.

11.7 Methoden aus Object: toString

In Kapitel 10 haben wir erwähnt, dass die universelle Superklasse **Object** einige Methoden implementiert, die somit alle Objekte anbieten. Die spannendste dieser Methoden ist **toString**, die wir jetzt vorstellen wollen. Wenn Sie weitere Details wissen möchten, können Sie sich die Schnittstelle von **Object** in der Dokumentation der Standardbibliothek ansehen.

Übung 11.4 Suchen Sie die Methode **toString** in der Java-Dokumentation. Welche Parameter hat sie? Welchen Ergebnistyp liefert sie?

Die Methode `toString` dient dazu, eine `String`-Repräsentation eines Objekts zu liefern. Das ist sehr nützlich für solche Objekte, die irgendwann einmal als Text in einer Benutzungsoberfläche sichtbar werden, hilft aber auch ganz allgemein bei allen Objekten – beispielsweise können sie während einer Fehlersuche gut ausgegeben werden.

Die Standardimplementierung von `toString` in der Klasse `Object` kann nicht sehr viele nützliche Details anzeigen. Wenn wir beispielsweise `toString` an einem `FotoEinsendung`-Objekt aufrufen, dann bekommen wir einen String in folgender Form:

FotoEinsendung@65c221c0

Das Ergebnis zeigt den Namen der Klasse des Objekts und eine interne Zahl.²

Konzept

Jedes Objekt in Java bietet eine Methode `toString` an, die eine `String`-Repräsentation des Objekts liefert. Um diese Methode nützlich zu machen, kann sie in Subklassen überschrieben werden.

Übung 11.5 Erzeugen Sie ein Objekt der Klasse `FotoEinsendung` in Ihrem Projekt und rufen Sie dann im Kontextmenü des Objekts die Methode `toString` aus dem Untermenü `GEERBT VON OBJECT` auf.

Typischerweise überschreiben wir diese Methode, um sie etwas nützlicher zu machen. Wir könnten beispielsweise die Methode `anzeigen` auf Basis der Methode `toString` definieren. Die Methode `toString` würde den Text nicht ausgeben, sondern ihn lediglich als Ergebnis liefern. Listing 11.3 zeigt, wie das aussehen könnte.

```
public class Einsendung
{
    ...

    public String toString()
    {
        String text = benutzername + "\n" + zeitString(zeitstempel);
        if(gefielWieOft > 0) {
            text += " - " + gefielWieOft +
                " Person(en) gefaellt dies.\n";
        }
        else {
            text += "\n";
        }

        if(kommentare.isEmpty()) {
            return text + " Keine Kommentare.\n";
        }
        else {
            return text + " " + kommentare.size() +
                " Kommentar(e). Zum Einsehen hier klicken.\n";
        }
    }

    public void anzeigen()
    {
        System.out.println(toString());
    }
}
```

Listing 11.3

`toString` in den Klassen `Einsendung` und `Nachrichteneinsendung`.

² Diese Nummer ist die Speicheradresse, an der dieses Objekt gespeichert ist. Sie ist nicht sehr nützlich und kann nur zur Identifikation dienen. Wenn diese Nummer bei zwei Aufrufen gleich ist, dann betrachten wir dasselbe Objekt. Wenn sie verschieden sind, haben wir auch unterschiedliche Objekte.

```

public class NachrichtenEinsendung extends Einsendung
{
    ...

    public String gibText()
    {
        return super.gibText() + nachricht + "\n";
    }

    public void anzeigen()
    {
        System.out.println(nachricht);
    }
}

```

Letztendlich würden wir versuchen, die **anzeigen**-Methoden komplett aus den Klassen zu verbannen. Der große Vorteil einer **toString**-Methode ist, dass wir nicht vorschreiben, was mit dem Text tatsächlich passiert. In der Originalversion unserer Anwendung wurde der Text immer auf der Konsole ausgegeben. Jetzt steht jedem Klienten (beispielsweise der Klasse **NewsFeed**) frei, wie er mit dem Text umgeht. Er kann ihn im Textbereich einer grafischen Benutzungsschnittstelle ausgeben, ihn in eine Datei speichern, ihn über ein Datennetz verschicken, ihn in einem Webbrowser anzeigen oder wie bisher, auf der Konsole ausgeben.

Die Anweisung zur Ausgabe einer Sendung könnte in einem Klienten dann folgendermaßen aussehen:

```
System.out.println(einsendung.toString());
```

Es geht aber noch eleganter, denn für die Methoden **System.out.print** und **System.out.println** gilt eine Besonderheit: Wenn der Parameter einer dieser Methoden nicht vom Typ **String** ist, dann wird in der Methode automatisch an dem übergebenen Objekt die Methode **toString** aufgerufen. Wir brauchen diesen Aufruf deshalb gar nicht selbst auszuführen, sondern können stattdessen schreiben:

```
System.out.println(einsendung);
```

Betrachten Sie nun die modifizierte Version der Methode **zeigen** aus der Klasse **NewsFeed** in Listing 11.4. In dieser Version haben wir den Aufruf von **toString** entfernt. Lässt sich dieser Quelltext übersetzen und fehlerfrei ausführen?

Listing 11.4

Eine neue Version der Methode **zeigen** in der Klasse **NewsFeed**.

```

public class NewsFeed
{
    // Datenfelder, Konstruktoren und andere Methoden hier ausgelassen

    /**
     * Gebe den Newsfeed aus. Derzeit werden die Details des Newsfeed auf die
     * Konsole ausgegeben. (Noch zu tun: ersetze diesen Code durch die Anzeige im
     * Webbrowser.)
     */
    public void zeigen()
    {
        // alle Einsendungen anzeigen
        for(Einsendung einsendung : einsendungen) {
            System.out.println(einsendung);
        }
    }
}

```

Tatsächlich *funktioniert* diese Methode wie erwartet. Wenn Sie dieses Beispiel im Detail erklären können, dann haben Sie die meisten Konzepte, die wir in diesem und dem vorigen Kapitel vorgestellt haben, anscheinend schon recht gut verstanden! Hier kommt eine ausführliche Erklärung der Ausgabeanweisung `println` innerhalb der Schleife:

- Die **for-each**-Schleife iteriert über alle Einsendungen und weist sie der Variablen mit dem statischen Typ **Einsendung** zu. Der dynamische Typ ist entweder **NachrichtenEinsendung** oder **FotoEinsendung**.
- Da dieses Objekt mit **System.out** ausgegeben wird und es kein **String** ist, wird automatisch seine **toString**-Methode aufgerufen.
- Der Aufruf dieser Methode ist nur deshalb zulässig, weil die Klasse **Einsendung** (der statische Typ!) eine **toString**-Methode hat. (Erinnern Sie sich: Die Typüberprüfung erfolgt mit dem statischen Typ. Dieser Aufruf wäre nicht zulässig, wenn die Klasse **Einsendung** nicht die Methode **toString** anbieten würde. Die Methode **toString** in der Klasse **Object** garantiert jedoch, dass jede Klasse diese Methode anbietet.)
- Die Ausgabe zeigt wie gewünscht alle Details an, weil alle möglichen dynamischen Typen (**NachrichtenEinsendung** und **FotoEinsendung**) die **toString**-Methode überschreiben und die dynamische Methodensuche gewährleistet, dass die jeweils redefinierende Methode aufgerufen wird.

Die Methode **toString** ist allgemein nützlich bei der Fehlersuche. Es ist oft angenehm, wenn Objekte sehr leicht vernünftig formatiert ausgegeben werden können. Die meisten Klassen aus der Java-Bibliothek überschreiben **toString** (alle Sammlungen beispielsweise können auf diese Weise ausgegeben werden) und für unsere selbst geschriebenen Klassen ist das meist auch sehr sinnvoll.

11.8 Objektgleichheit: equals und hashCode

In der Praxis stellt sich häufig die Frage, ob zwei Objekte „gleich“ sind. Hierzu definiert die Klasse **Object** die beiden Methoden **equals** und **hashCode**, die sehr eng mit der Frage nach Gleichheit verknüpft sind. Allerdings sollten wir mit dem Begriff „Gleichheit“ sehr sorgfältig umgehen, da er im Zusammenhang mit Objekten zwei ganz verschiedene Dinge bezeichnen kann. Manchmal möchten wir wissen, ob zwei verschiedene Variablen auf dasselbe Objekt verweisen. Ein solcher Fall liegt beispielsweise vor, wenn eine Objektvariable einer Methode als Parameter übergeben wird: Es gibt nur ein Objekt, aber sowohl die ursprüngliche Variable als auch die Parametervariable verweisen darauf. Gleiches gilt, wenn eine beliebige Objektvariable einer anderen zugewiesen wird. Situationen wie diese führen zu einer sogenannten *Referenzgleichheit*. Ob Referenzgleichheit vorliegt, wird mit dem **==**-Operator überprüft. Somit liefert der folgende Test **true**, wenn **var1** und **var2** auf dasselbe Objekt verweisen (oder beide **null** sind), und **false**, wenn jede Variable auf etwas anderes verweist:

```
var1 == var2
```

Bei der Referenzgleichheit bleiben die *Inhalte* der Objekte, auf die Bezug genommen wird, völlig unberücksichtigt. Das einzige, was zählt, ist, ob zwei verschiedene Variablen auf ein Objekt oder auf zwei verschiedene Objekte verweisen.

Aus diesem Grunde definieren wir neben der Referenzgleichheit die sogenannte *Inhaltsgleichheit*. Wenn der Inhalt auf Gleichheit geprüft wird, wird untersucht, ob zwei Objekte intern gleich sind, das heißt, ob die inneren Zustände von zwei Objekten gleich sind. Das war der Grund, warum wir für den Vergleich von Zeichenketten in Kapitel 6 nicht die Referenzgleichheit verwendet haben.

Was unter Inhaltsgleichheit zwischen zwei Objekten zu verstehen ist, wird von der Klasse der Objekte definiert. Genau dort machen wir Gebrauch von der **equals**-Methode, die jede Klasse von der Superklasse **Object** erbt. Wenn wir definieren müssen, was es für zwei Objekte bedeutet, hinsichtlich ihrer internen Zustände gleich zu sein, müssen wir die **equals**-Methode überschreiben, die es gestattet, zwei Objekte wie folgt zu vergleichen:

```
var1.equals(var2)
```

Wir müssen sie überschreiben, weil die von der Klasse **Object** geerbte **equals**-Methode nur auf Referenzgleichheit prüft. Sie sieht in etwa wie folgt aus:

```
public boolean equals(Object obj)
{
    return this == obj;
}
```

Da die Klasse **Object** weder über eigene Datenfelder verfügt noch vorausahnen kann, welche Datenfelder in den Subklassen vorhanden sind, gibt es keinen Zustand, den sie vergleichen könnte.

Um festzustellen, ob zwei Objekte inhaltsgleich sind, müssen Sie prüfen, ob die Werte ihrer beiden Datenfeldsätze gleich sind. Beachten Sie jedoch, dass der Parameter der **equals**-Methode vom Typ **Object** ist, sodass eine Prüfung der Datenfelder nur dann sinnvoll ist, wenn Sie Datenfelder des gleichen Typs vergleichen. Das bedeutet, dass wir erst einmal ermitteln müssen, ob der Typ des als Parameter übergebenen Objekts mit dem Typ des Objekts übereinstimmt, mit dem es verglichen wird. So könnten wir beispielsweise die Methode in der Klasse **Student** aus unserem Projekt *Laborkurs* aus Kapitel 1 wie folgt schreiben:

```
public boolean equals(Object obj)
{
    if(this == obj) {
        return true; // Referenzgleichheit
    }
    if(!(obj instanceof Student)) {
        return false; // nicht derselbe Typ
    }
    // Zugriff auf die Datenfelder des anderen Studenten
    Student anderer = (Student) obj;
    return name.equals(anderer.name) &&
        matrikelnummer.equals(anderer.matrikelnummer) &&
        schein == anderer.schein;
}
```

Die erste Prüfung dient lediglich der Effizienzsteigerung. Wenn dem Objekt für den Vergleich eine Referenz auf sich selbst übergeben wurde, dann wissen wir, dass Inhaltsgleichheit vorliegen muss. Die zweite Prüfung stellt sicher, dass wir zwei Studenten vergleichen. Wenn nicht, betrachten wir die beiden Objekte

automatisch als verschieden. Nachdem wir festgestellt haben, dass das andere Objekt vom Typ **Student** ist, bedienen wir uns des Cast-Operators und einer weiteren Variable des richtigen Typs, auf deren Details wir nachfolgend problemlos zugreifen können. Und schließlich nutzen wir die Tatsache, dass die Instanz einer Klasse direkt auf die privaten Elemente eines anderen Objekts dieser Klasse zugreifen kann. Dies ist in diesem Falle sehr wichtig, da nicht immer für jedes private Datenfeld in einer Klasse sondierende Methoden definiert sind. Beachten Sie, dass wir die Instanzfelder **matrikelnummer** und **name** durchweg auf inhaltliche Gleichheit geprüft haben und nicht auf Referenzgleichheit.

Es ist nicht immer notwendig, alle Datenfelder der beiden Objekte zu vergleichen, um festzustellen, ob diese gleich sind. Wenn wir beispielsweise mit Sicherheit wissen, dass jedem **Student** eine eindeutige **matrikelnummer** zugewiesen wurde, müssen wir die Datenfelder **name** und **scheine** nicht überprüfen. Dann wäre es möglich, die letzte Anweisung wie folgt zu verkürzen:

```
return matrikelnummer.equals(anderer.matrikelnummer);
```

Immer, wenn die **equals**-Methode überschrieben wird, muss die **hashCode**-Methode ebenfalls überschrieben werden. Die **hashCode**-Methode wird von Datenstrukturen wie **HashMap** und **HashSet** verwendet, um ein effizientes Einfügen und Suchen von Objekten in diesen Sammlungen zu gewährleisten. Im Wesentlichen liefert die **hashCode**-Methode einen ganzzahligen Wert zurück, der ein Objekt repräsentiert. Die Standardimplementierung in der Klasse **Object** versucht für ungleiche Objekte auch unterschiedliche **hashCode**-Werte zurückzuliefern.

Die beiden Methoden **equals** und **hashCode** sind insofern eng miteinander verbunden, als zwei Objekte, die gemäß **equals** gleich sind, auch identische **hashCode**-Werte zurückliefern müssen. Diese Forderung bzw. Übereinkunft ist in der Beschreibung zu **hashCode** in der API-Dokumentation der Klasse **Object** zu finden.³ Es würde den Rahmen dieses Buches sprengen, wenn wir detailliert beschreiben würden, wie man am besten Hashcodes berechnet. Doch dem interessierten Leser möchten wir das Werk von Joshua Bloch *Effective Java* ans Herz legen, dessen Technik wir hier einsetzen.⁴ Letzten Endes geht es darum, auf der Basis der Werte der Datenfelder, die von der überschriebenen **equals**-Methode verglichen werden, einen ganzzahligen Wert zu berechnen. Sehen Sie im Folgenden eine hypothetische **hashCode**-Methode, die die Werte eines **int**-Feldes namens **zaehler** sowie ein **String**-Datenfeld namens **name** nutzt, um den Code zu berechnen:

```
public int hashCode()
{
    int ergebnis = 17; // ein beliebiger Anfangswert
    // Sorge dafür, dass der berechnete Wert von der Reihenfolge
    // abhängt, in der die Datenfelder verarbeitet werden
    ergebnis = 37 * ergebnis + zaehler;
    ergebnis = 37 * ergebnis + name.hashCode();
    return ergebnis;
}
```

³ Beachten Sie, dass es auf der anderen Seite nicht notwendig ist, dass ungleiche Objekte auch immer ungleiche Hashcodes zurückliefern.

⁴ Joshua Bloch, *Effective Java*, 2. Auflage, Addison-Wesley, 2008.

11.9 Der Zugriff über `protected`

Konzept

Datenfelder und Methoden, die als **protected** deklariert sind, sind für (direkte und indirekte) Subklassen zugreifbar.

In Kapitel 10 haben wir angemerkt, dass die Regeln über öffentliche und private Sichtbarkeit von Klasseneigenschaften sowohl zwischen einer Subklasse und ihrer Superklasse als auch zwischen Klassen aus verschiedenen Vererbungshierarchien gelten. Das kann manchmal etwas restriktiv sein, denn das Verhältnis zwischen einer Superklasse und ihren Subklassen ist deutlich enger als das zwischen anderen Klassen. Aus diesem Grund definieren objektorientierte Sprachen oft eine Zugriffsstufe, die zwischen komplett privatem Zugriff und freier öffentlicher Verfügbarkeit liegt. In Java wird diese Stufe *protected* genannt und entsprechend durch das Schlüsselwort **protected** definiert (als Alternative zu **private** und **public**). Listing 11.5 zeigt ein Beispiel einer sondierenden Methode für die Klasse **Einsendung**, die als **protected** deklariert ist.

Listing 11.5

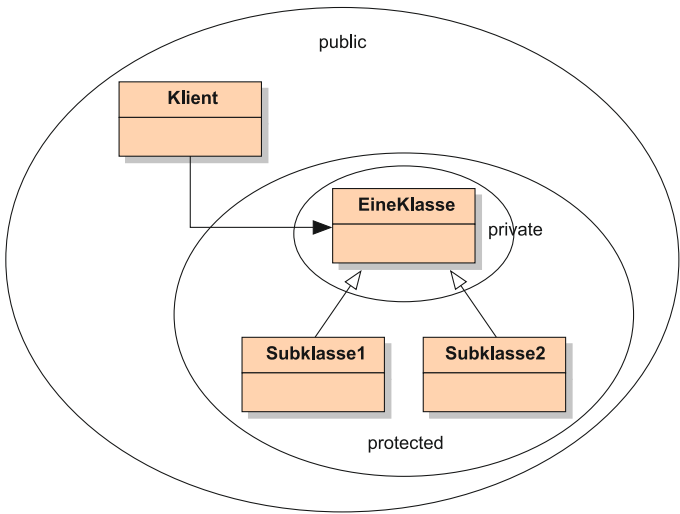
Ein Beispiel für eine als **protected** deklarierte Methode.

```
protected long gibZeitstempel()  
{  
    return zeitstempel;  
}
```

Datenfelder und Methoden mit der Zugriffsstufe **protected** können innerhalb der Klasse selbst und von allen Subklassen benutzt werden, aber nicht von anderen Klassen.⁵ Die Methode **gibZeitstempel** aus Listing 11.5 kann in der Klasse **Einsendung** oder in jeder ihrer Subklassen aufgerufen werden, aber nicht in anderen Klassen. Abbildung 11.8 illustriert dies: Die ovalen Bereiche in dem Diagramm kennzeichnen jeweils die Gruppe der Klassen, die auf Teile der Klasse **EineKlasse** zugreifen können.

Abbildung 11.8

Zugriffsstufen:
private, **protected**
und **public**.



Obwohl alle Teile einer Klasse als **protected** deklariert werden können, sollte dies üblicherweise den Konstruktoren und Methoden vorbehalten bleiben. Datenfelder

⁵ In Java ist diese Regel nicht so eindeutig, wie hier beschrieben, da Java eine weitere Sichtbarkeitsstufe kennt, die *Paketstufe* genannt wird, aber nicht mit einem bestimmten Schlüsselwort verbunden ist. Wir werden hierauf nicht näher eingehen und Sie sollten im Allgemeinen davon ausgehen, dass die Zugriffsstufe **protected** der besonderen Beziehung zwischen Superklasse und Subklasse vorbehalten ist.

werden üblicherweise nicht als `protected` deklariert, da dies die Kapselung schwächen würde. Soweit möglich, sollten veränderbare Datenfelder in Superklassen privat bleiben. Es kann allerdings vereinzelte Ausnahmen geben, in denen der direkte Zugriff auf Datenfelder aus Subklassen sinnvoll ist. Durch Vererbung wird eine sehr viel engere Kopplung zwischen Klassen definiert als durch Klientenbeziehungen.

Vererbung bindet Klassen stärker aneinander und eine Veränderung an einer Superklasse kann leichter zu Problemen in einer Subklasse führen. Dies sollte beim Entwurf von Klassen und ihren Beziehungen berücksichtigt werden.

Übung 11.6 Die Version der Methode `anzeigen` aus Listing 11.2 erzeugt die in Abbildung 11.9 gezeigte Ausgabe. Ordnen Sie in Ihrer Version des `Netzwerk`-Projekts die Anweisungen in der Methode so um, dass die Details wie in Abbildung 11.10 ausgegeben werden.

```
Leonardo da Vinci
vor 40 Sekunden. 2 Person(en) gefällt dies.
Keine Kommentare
```

```
Hatte heute Morgen eine grossartige Idee.
Doch jetzt habe ich vergessen, was es war. Es hatte irgendwas mit
Fliegen zu tun ...
```

Abbildung 11.9

Mögliche Ausgabe von `anzeigen`: Aufruf der Superklasse zu Anfang von `anzeigen` (markierte Bereiche werden von der Superklasse ausgegeben).

```
Hatte heute Morgen eine grossartige Idee.
Doch jetzt habe ich vergessen, was es war. Es hatte irgendwas mit
Fliegen zu tun ...
```

```
Leonardo da Vinci
vor 40 Sekunden. 2 Person(en) gefällt dies.
Keine Kommentare
```

Abbildung 11.10

Alternative Ausgabe von `anzeigen` (markierte Bereiche werden von der Superklasse ausgegeben).

Übung 11.7 Die Notwendigkeit, in der Methode `anzeigen` den `super`-Aufruf durchführen zu müssen, schränkt uns etwas bei der Formatierung der Ausgabe ein, denn wir sind abhängig davon, wie die Superklasse ihre Daten formatiert. Nehmen Sie die notwendigen Änderungen an der Klasse `Einsendung` und an der Methode `anzeigen` in `NachrichtenEinsendung` vor, sodass die Ausgabe aus Abbildung 11.11 produziert wird. Alle Änderungen an der Klasse `Einsendung` sollten nur für ihre Subklassen sichtbar sein. *Hinweis:* Um dies zu erreichen, könnten Sie der Klasse sondierende `protected`-Methoden hinzufügen.

Abbildung 11.11

Ausgabe von **anzeigen**, in der Details der Super- und Subklasse gemischt sind (markierte Bereiche kennzeichnen Daten der Superklasse).

Leonardo da Vinci

Hatte heute Morgen eine grossartige Idee.
Doch jetzt habe ich vergessen, was es war. Es hatte irgendwas mit Fliegen zu tun ...

vor 40 Sekunden. 2 Person(en) gefällt dies.
Keine Kommentare

11.10 Der Operator instanceof

Die Einführung der Vererbung in das *Netzwerk*-Projekt hatte unter anderem zur Folge, dass die **NewsFeed**-Klasse nur **Einsendung**-Objekte kennt und nicht zwischen Nachrichten- und Fotoeinsendungen unterscheiden kann. Dies bot die Möglichkeit, alle Arten von Einsendungen in einer einzelnen Liste zu speichern.

Doch was wäre, wenn wir nur die Nachrichteneinsendungen oder nur die Fotoeinsendungen aus der Liste herausziehen möchten? Wie würden wir dann vorgehen? Oder nehmen wir an, wir suchten nach einer Nachricht von einem bestimmten Autor. Würde die Klasse **Einsendung** eine Methode **gibAutor** definieren, wäre dies an sich kein großes Problem, doch es bliebe dann immer noch die Frage, wie wir reagieren sollen, wenn einige der Nachrichteneinsendungen und Fotoeinsendungen denselben Autor haben. Macht es dann einen Unterschied, welcher Typ zurückgeliefert wird?

Es gibt Fälle, in denen wir den speziellen dynamischen Typ eines Objekts ermitteln müssen und uns der gemeinsame Supertyp nicht ausreicht. Hierfür stellt uns Java den **instanceof**-Operator zur Verfügung. Er prüft, ob ein gegebenes Objekt direkt oder indirekt eine Instanz einer gegebenen Klasse ist. Der Test

```
obj instanceof MeineKlasse
```

liefert **true** zurück, wenn der dynamische Typ von **obj** gleich **MeineKlasse** oder einer Subklasse von **MeineKlasse** ist. Der linke Operand ist immer eine Objektreferenz und der rechte Operand immer der Name einer Klasse. So liefert zum Beispiel

```
einsendung instanceof NachrichtenEinsendung
```

nur dann **true** zurück, wenn **einsendung** eine **NachrichtenEinsendung** und keine **FotoEinsendung** ist.

Nach dem Einsatz des **instanceof**-Operators folgt oft direkt eine Typumwandlung der Objektreferenz in den identifizierten Typ – wie in dem folgenden Code, der alle Nachrichteneinsendungen aus einer Liste der Einsendungen herausfiltert und in einer eigenen Liste speichert:

```
ArrayList<NachrichtenEinsendung> nachrichten =
    new ArrayList<>();
for(Einsendung einsendung : einsendungen) {
    if(einsendung instanceof NachrichtenEinsendung) {
        nachrichten.add(einsendung);
    }
}
```

Es sollte jedoch klar sein, dass die Typumwandlung das Einsendungsobjekt in keiner Weise ändert, da wir gerade festgestellt haben, dass es bereits ein **NachrichtenEinsendung**-Objekt ist.

11.11 Ein weiteres Beispiel für Vererbung mit Überschreiben

Um ein weiteres Beispiel mit einer ähnlichen Verwendung von Vererbung zu diskutieren, gehen wir zurück zum *Zuul*-Projekt aus Kapitel 8. In dem Spiel *Die Welt von Zuul* haben wir eine Reihe von **Raum**-Objekten benutzt, um eine Umgebung für ein einfaches Spiel zu erzeugen. **Übung 8.45** schlug vor, dass Sie einen Teleporter-Raum implementieren (einen Raum, der den Spieler in einen zufälligen anderen Raum „beamt“, wenn er ihn betritt oder verlässt). Wir betrachten diese Übung hier noch einmal, weil ihre Lösung sehr stark von Vererbung profitieren kann. Wenn Sie sich nicht mehr an das Projekt erinnern, schlagen Sie noch einmal kurz in Kapitel 8 nach oder sehen Sie sich Ihr eigenes *Zuul*-Projekt an.

Es gibt nicht nur eine Lösung für diese Aufgabe. Viele verschiedene Ansätze sind denkbar und können implementiert werden. Einige von ihnen können aber durchaus besser sein als andere – eleganter, einfacher zu lesen, einfacher zu warten oder einfacher zu erweitern.

Nehmen Sie an, dass wir diese Aufgabe so implementieren wollen, dass der Spieler automatisch in einen zufällig gewählten Raum gebeamt wird, wenn er den Teleporter-Raum verlassen will. Die offensichtlichste Lösung, die den meisten unmittelbar in den Sinn kommt, implementiert dieses Verhalten in der Klasse **Spiel**, da dort die Befehle implementiert sind. Einer der Befehle ist **go**, der in der Methode **wechsleRaum** implementiert ist. In dieser Methode ist die folgende Anweisung der zentrale Quelltextabschnitt:

```
naechsterRaum = aktuellerRaum.gibAusgang(richtung);
```

Diese Anweisung fragt vom aktuellen Raum den Nachbarraum in der angegebenen Richtung ab. Um unsere Teleportation einzubauen, könnten wir diese Anweisung folgendermaßen modifizieren:

```
if (aktuellerRaum.gibName().equals("Teleporter-Raum")) {
    naechsterRaum = gibZufaellichenRaum();
}
else {
    naechsterRaum = aktuellerRaum.gibAusgang(richtung);
}
```

Die Idee ist einfach: Wir prüfen, ob wir im Teleporter-Raum sind. Wenn ja, dann lassen wir den nächsten Raum zufällig auswählen (wir müssen die Methode **gibZufaellichenRaum** natürlich noch implementieren), ansonsten machen wir das Gleiche wie vorher.

Obwohl diese Lösung funktioniert, hat sie einige Nachteile. Der erste ist, dass der Raum über seinen Namen identifiziert wird. Nehmen Sie an, jemand wollte dieses Spiel in eine andere Sprache übersetzen, beispielsweise ins Englische. Dann würde

der Raum nicht mehr „Teleporter-Raum“ genannt, sondern vielleicht „teleporting room“ – und das Spiel würde nicht mehr funktionieren! Das ist ganz eindeutig ein Wartungsproblem.

Die zweite, etwas bessere Lösung würde deshalb eine Instanzvariable anstelle des Raumnamens verwenden, um den Teleporter-Raum zu identifizieren. Ungefähr so:

```
if(aktuellerRaum == teleporterRaum) {
    naechsterRaum = gibZufaelligenRaum();
}
else {
    naechsterRaum = aktuellerRaum.gibAusgang(richtung);
}
```

Hier nehmen wir an, dass es eine Instanzvariable **teleporterRaum** vom Typ **Raum** gibt, in der wir die Referenz auf unseren Teleporter-Raum speichern.⁶ Die Überprüfung ist nun unabhängig vom Namen des Raumes. Das ist etwas besser.

Es gibt aber immer noch Raum für Verbesserungen. Wir erkennen die Nachteile dieser Lösung, wenn wir an eine weitere Wartungsänderung denken. Nehmen Sie an, wir möchten zwei weitere Teleporter-Räume hinzufügen, sodass das Spiel über drei verschiedene Teleporter-Räume verfügt.

Ein sehr schöner Aspekt unseres bestehenden Entwurfs war, dass wir den Raumplan an einer einzigen Stelle festlegen konnten und der Rest des Spiels von dieser Planung komplett unabhängig war. Wir konnten das Layout der Räume sehr leicht ändern und trotzdem hat alles noch funktioniert – ein Muster an Wartbarkeit! Mit unserer aktuellen Erweiterung geht dieser Vorteil jedoch verloren. Wenn wir zwei weitere Teleporter-Räume hinzufügen, dann müssen wir zwei weitere Instanzvariablen oder ein Array (für die Referenzen auf die Teleporter-Räume) definieren und wir müssen die Methode **wechsleRaum** ändern, damit diese auch auf die beiden neuen Räume abprüft. Aus Sicht von einfacher Änderbarkeit ist das ein Schritt rückwärts.

Die entscheidende Frage ist also: Wie können wir eine Lösung finden, bei der nicht bei jedem neuen Teleporter-Raum die Implementierung eines Befehls geändert werden muss? Hier kommt die nächste Idee.

Wir können in der Klasse **Raum** eine Methode **istTeleporterRaum** hinzufügen. Auf diese Weise muss das **Spiel**-Objekt nicht alle Teleporter-Räume kennen – die Räume selbst können diese Information halten. Wenn ein Raum erzeugt wird, dann kann über einen booleschen Wert entschieden werden, ob es sich um einen Teleporter-Raum handelt oder nicht. Die Methode **wechsleRaum** könnte dann den folgenden Code verwenden:

```
if(aktuellerRaum.istTeleporterRaum()) {
    naechsterRaum = gibZufaelligenRaum();
}
else {
    naechsterRaum = aktuellerRaum.gibAusgang(richtung);
}
```

⁶ Überlegen Sie sich, warum es hier am sinnvollsten ist, auf Referenzgleichheit zu testen.

Nun können wir so viele Teleporter-Räume hinzufügen, wie wir wollen – die Klasse **Spiel** muss nicht jedes Mal angepasst werden. Allerdings hat die Klasse **Raum** hierzu ein neues Datenfeld bekommen, das eigentlich nur für ein oder zwei Instanzen benötigt wird. Solcher Spezialcode ist immer ein Hinweis darauf, dass wir es mit einem schlechten Entwurf zu tun haben. Dieser Ansatz ist auch nicht gut erweiterbar für den Fall, dass wir weitere Spezialräume einführen wollen, von denen jeder seine eigenen Datenfelder und sondierenden Methoden benötigt.⁷

Mithilfe von Vererbung können wir das besser machen und eine Lösung entwerfen, die sogar noch flexibler als die letzte ist. Wir können eine Klasse **Teleporter-Raum** als Subklasse der Klasse **Raum** implementieren. In dieser neuen Klasse können wir die Methode **gibAusgang** überschreiben und ihre Implementierung so ändern, dass sie einen zufällig gewählten Raum zurückliefert:

```
public class TeleporterRaum extends Raum
{
    /**
     * Liefere einen zufällig gewählten Raum, unabhängig vom Parameter richtung.
     * @param richtung wird ignoriert.
     * @return einen zufälligen Ausgang aus diesem Raum.
     */
    public Raum gibAusgang(String richtung)
    {
        return findeZufaellichenRaum();
    }

    /**
     * Wähle zufällig einen Raum aus.
     */
    private Raum findeZufaellichenRaum()
    {
        ... // Implementierung hier ausgelassen
    }
}
```

Diese Lösung ist ausgesprochen elegant, da weder die ursprüngliche Klasse **Spiel** noch die Klasse **Raum** geändert werden muss! Wir können diese Klasse einfach dem Projekt hinzufügen und die Methode **wechsleRaum** funktioniert genau wie vorher. Das Einfügen des Teleporter-Raums bei der Erzeugung der Räume und ihrer Verbindungen ist (fast) alles, was zu tun ist. Beachten Sie auch, dass die neue Klasse keine Markierung benötigt, die ihren Sonderstatus signalisiert – ihr Typ und ihr spezifisches Verhalten sorgen automatisch für diese Unterscheidung.

Weil **TeleporterRaum** eine Subklasse von **Raum** ist, können Objekte dieser Klasse an allen Stellen verwendet werden, an denen **Raum**-Objekte erwartet werden. Sie können als Nachbarräume für andere Räume dienen und in der Klasse **Spiel** als aktueller Raum gehalten werden.

Wir haben hier die Implementierung der Methode **findeZufaellichenRaum** ausgelassen. In der Praxis wäre es vermutlich besser, diese Funktionalität in einer eigenen Klasse (etwa **ZufallsraumWaehler**) anzubieten als in der Klasse **TeleporterRaum** selbst. Wir überlassen diese Entscheidung dem Leser zur Übung.

⁷ Wir könnten an dieser Stelle auch **instanceof** verwenden, aber keine dieser Lösungen wäre wirklich gut.

Übung 11.8 Implementieren Sie einen **Teleporter**-Raum mithilfe von Vererbung in Ihrer Version des *Zuul*-Projekts.

Übung 11.9 Diskutieren Sie, wie man mithilfe von Vererbung im *Zuul*-Projekt eine **Spieler**- und eine **Monster**-Klasse implementieren könnte.

Übung 11.10 Könnte (oder sollte) Vererbung benutzt werden, um eine Vererbungsbeziehung (Super-, Sub- oder Geschwisterklasse) zwischen einer Figur des Spiels und einem Gegenstand zu definieren?

Zusammenfassung



Wenn wir mit Subklassen und polymorphen Variablen zu tun haben, dann müssen wir zwischen dem statischen und dem dynamischen Typ einer Variablen unterscheiden. Der statische Typ ist der deklarierte Typ, während der dynamische Typ der Typ des Objekts ist, auf das die Variable jeweils verweist.

Typüberprüfungen finden auf Basis des statischen Typs statt, während die Methodensuche zur Laufzeit auf dem dynamischen Typ basiert. Dies ermöglicht uns, sehr flexible Strukturen durch Überschreiben von Methoden zu erzeugen. Auch wenn wir über eine Variable eines Supertyps einen Aufruf durchführen, garantiert uns das Überschreiben, dass die spezialisierten Methoden des jeweiligen Subtyps aufgerufen werden. Auf diese Weise können Objekte verschiedener Klassen unterschiedlich auf denselben Methodenaufruf reagieren.

Wenn wir überschreibende Methoden implementieren, können wir mit dem Schlüsselwort **super** die überschriebene Version der Superklasse aufrufen. Wenn Datenfelder und Methoden mit dem Zugriffsmodifikator **protected** deklariert werden, sind sie für Subklassen zugreifbar, für andere Klassen jedoch nicht.

NEUE BEGRIFFE IN DIESEM KAPITEL

statischer Typ, dynamischer Typ, Überschreiben, Redefinition, Methodensuche, Methodenauswahl, Methoden-Polymorphie, **protected**

Zusammenfassung der Konzepte

- **Statischer Typ** Der statische Typ einer Variablen **v** ist der Typ, mit dem die Variable im Quelltext der Klasse deklariert wurde.
- **Dynamischer Typ** Der dynamische Typ einer Variablen **v** ist der Typ des Objekts, das zurzeit in der Variablen **v** gehalten wird.
- **Überschreiben einer Methode** Eine Subklasse kann die Implementierung einer Methode überschreiben. Dazu deklariert die Subklasse eine Methode mit der gleichen Signatur wie in der Superklasse, implementiert diese jedoch mit einem anderen Rumpf. Die überschreibende Methode wird dann bei Aufrufen an Objekte der Unterklasse vorgezogen.
- **Methoden-Polymorphie** Methodenaufrufe in Java sind polymorph. Derselbe Methodenaufruf kann zu unterschiedlichen Zeitpunkten verschiedene Methoden aufrufen, abhängig vom dynamischen Typ der Variablen, mit der der Aufruf durchgeführt wird.
- **toString** Jedes Objekt in Java bietet eine Methode **toString** an, die eine **String**-Repräsentation des Objekts liefert. Um diese Methode nützlich zu machen, kann sie in Subklassen überschrieben werden.
- **protected** Datenfelder und Methoden, die als **protected** deklariert sind, sind für (direkte und indirekte) Subklassen zugreifbar.



Übung 11.11 Nehmen Sie an, Sie sehen folgende Quelltextzeilen:

```
Geraet geraet = new Drucker();
geraet.gibName();
```

Drucker ist eine Subklasse von **Geraet**. Welche dieser Klassen muss eine Methode **gibName** definieren, damit sich diese Zeilen übersetzen lassen?

Übung 11.12 In der gleichen Situation wie in der vorigen Übung: Wenn beide Klassen eine Implementierung von **gibName** enthalten, welche wird ausgeführt?

Übung 11.13 Nehmen Sie an, Sie schreiben eine Klasse **Student**, die keine explizit definierte Superklasse hat. Sie schreiben keine **toString**-Methode. Betrachten Sie die folgenden Quelltextzeilen:

```
Student st = new Student();
String s = st.toString();
```

Lassen sich diese Zeilen übersetzen? Wenn ja, was passiert, wenn Sie sie ausführen?

Übung 11.14 In der gleichen Situation wie in der vorigen Übung (Klasse **Student**, keine Methode **toString**): Lassen sich die folgenden Zeilen übersetzen? Warum?

```
Student st = new Student();
System.out.println(st);
```

Übung 11.15 Nehmen Sie an, Ihre Klasse **Student** überschreibt **toString** so, dass die Methode den Namen des Studenten liefert. Sie haben weiterhin eine Liste von Studenten. Lassen sich die folgenden Zeilen übersetzen? Wenn nicht, warum? Wenn ja, was wird ausgegeben? Erklären Sie im Detail, was passiert.

```
for (Object st : meineListe) {
    System.out.println(st);
}
```

Übung 11.16 Schreiben Sie einige Zeilen Quelltext, die dazu führen, dass eine Variable **x** den statischen Typ **T** und den dynamischen Typ **D** hat.

KAPITEL

12

Weitere Techniken zur Abstraktion



Lernziele

Zentrale Konzepte in diesem Kapitel: abstrakte Klassen, Interfaces, multiple Vererbung

Java-Konstrukte in diesem Kapitel: `abstract`, `implements`, `interface`

In diesem Kapitel untersuchen wir weitere Techniken, die mithilfe von Vererbungskonzepten Entwürfe verbessern und sie wartbarer und erweiterbarer machen. Diese Techniken führen Verbesserungen ein, mit denen Abstraktionen besser repräsentiert werden können.

In den beiden vorherigen Kapiteln wurden die wichtigsten Aspekte von Vererbung für den Entwurf von Anwendungen vorgestellt, aber einige fortgeschrittene Benutzungen und Probleme wurden bisher nicht betrachtet. Wir werden das Bild in diesem Kapitel mit einem etwas anspruchsvolleren Beispiel abrunden.

Das Projekt, das wir in diesem Kapitel benutzen werden, ist eine Simulation. Wir setzen es zur Diskussion von Vererbung ein und demonstrieren, dass einige für uns neue Probleme auftreten. Daraufhin werden abstrakte Klassen und Interfaces eingeführt, mit denen diese Probleme gelöst werden können.

12.1 Simulationen

Sehr häufig werden Computer eingesetzt, um Simulationen von realen Systemen durchzuführen. Dies schließt Verkehrssimulationen, Wettervorhersagen, Simulationen von Atomreaktionen, Börsensimulationen, Umweltsimulationen und vieles mehr ein. Tatsächlich werden weltweit die meisten Supercomputer für Simulationen eingesetzt.

Wenn wir eine Computersimulation entwerfen, versuchen wir das Verhalten eines Ausschnitts aus der realen Welt in einem Softwaremodell zu erfassen. Jede Simulation nimmt dabei zwangsläufig Vereinfachungen vor. Die Entscheidung, welche Details vernachlässigt werden können und welche berücksichtigt werden sollten, ist oft sehr anspruchsvoll. Je detaillierter eine Simulation ist, desto genauer kann sie das Verhalten des realen Systems vorhersagen. Aber mehr Details erhöhen nicht